

Environment and Foundations

Setting up the stack, and the physics that makes the rest of the project meaningful

Phase 0 installs nothing conceptual and everything practical — and then spends most of its length on the one idea the entire project rests on: the Born-Oppenheimer approximation, which is what allows a molecule's energy to be treated as a function of its geometry.

Estimated time: **30 minutes** for the install, an hour if you read the theory properly. Do read the theory properly.

In this document

Where this phase sits, and an honest statement of what the project does and does not demonstrate

The physics: ground states, atomic units, Born-Oppenheimer, and the potential energy surface

The six-stage pipeline every energy evaluation passes through

Step-by-step environment build with a verification script that computes real energies

Reference targets for the whole project, and the traps that follow from a minimal basis

1. Where this phase sits

This is the orientation phase. By the end of it you will have a working Python environment, a verification script that proves every library imports and produces a correct number, and — more importantly — a clear mental model of what the other six phases are actually doing. No quantum code is written here. That is deliberate.

The project as a whole computes the **bond dissociation curve of the hydrogen molecule** using a Variational Quantum Eigensolver, and checks that curve against two classical references. The finished artefact is a single figure with three overlaid curves, three extracted physical constants, and a short written analysis.

What you will have at the end of Phase 0

A virtual environment with PennyLane, PySCF, NumPy, SciPy and Matplotlib, all importing cleanly.

A `verify_env.py` script that computes a real Hartree-Fock energy and prints a known-correct value.

A project directory laid out so later phases have somewhere to put their code and data.

An understanding of the Born-Oppenheimer approximation, which is the assumption that makes the entire project meaningful.

2. Read this before you build anything

For H_2 in a minimal STO-3G basis, VQE with a UCCSD ansatz reproduces the exact full-configuration-interaction result for that basis. The quantum algorithm gives you nothing a classical solver cannot produce in milliseconds on the same laptop.

That is expected, and it is fine. But it means **the deliverable is not quantum advantage**. The deliverable is a *verified implementation*: you plot the VQE curve and the exact curve on the same axes and demonstrate that they lie on top of one another. A project that states this plainly is stronger — and far more credible to anyone who knows the field — than one that overclaims.

The honest framing, in one sentence

"I implemented the full quantum-chemistry pipeline from geometry to qubit Hamiltonian to variational solution, and verified it against exact diagonalisation at every geometry." That is a real claim about real work. "I used a quantum computer to solve chemistry" is not.

3. The physics you need before any code

3.1 What a ground state is, and why we want it

A molecule's electrons can be arranged in many configurations, each with its own energy. The **ground state** is the lowest-energy arrangement. It determines whether the molecule is stable, how long its bonds are, how stiff they are, and how much energy is released when it forms. Essentially every question in chemistry reduces to finding ground-state energies.

Formally, we want the lowest **eigenvalue** of the molecular Hamiltonian — the operator that encodes all the kinetic and electrostatic energy in the system. Every method in this project (Hartree-Fock, FCI, VQE) is a different strategy for getting at that one number.

3.2 Units: Hartree, Bohr, and chemical accuracy

Hartree (Ha)	The atomic unit of energy. $1 \text{ Ha} \approx 27.211 \text{ eV} \approx 2625.5 \text{ kJ/mol}$. All energies in this project are reported in Hartree.
Millihartree (mHa)	One thousandth of a Hartree. The natural unit for <i>errors</i> in quantum chemistry.
Bohr (a_0)	The atomic unit of length. $1 \text{ Å} = 1.8897 \text{ Bohr}$. PennyLane's chemistry module defaults to Bohr; you almost certainly think in Ångström. This mismatch is the single most expensive bug in the project.
Chemical accuracy	$1 \text{ kcal/mol} \approx 1.6 \text{ mHa}$. The threshold below which a calculation becomes predictively useful for chemistry. Your VQE should beat this by orders of magnitude for H_2 .

3.3 The Born-Oppenheimer approximation

Nuclei are roughly 1836 times heavier than electrons. Both feel Coulomb forces of the same magnitude, so by $a = F/m$ the nuclei accelerate about 1836 times less. Working the numbers through for H_2 : electrons circulate on a timescale of $\sim 0.15 \text{ fs}$ while the molecule vibrates on a timescale of $\sim 7.6 \text{ fs}$, so the electrons complete around fifty full cycles per single nuclear vibration.

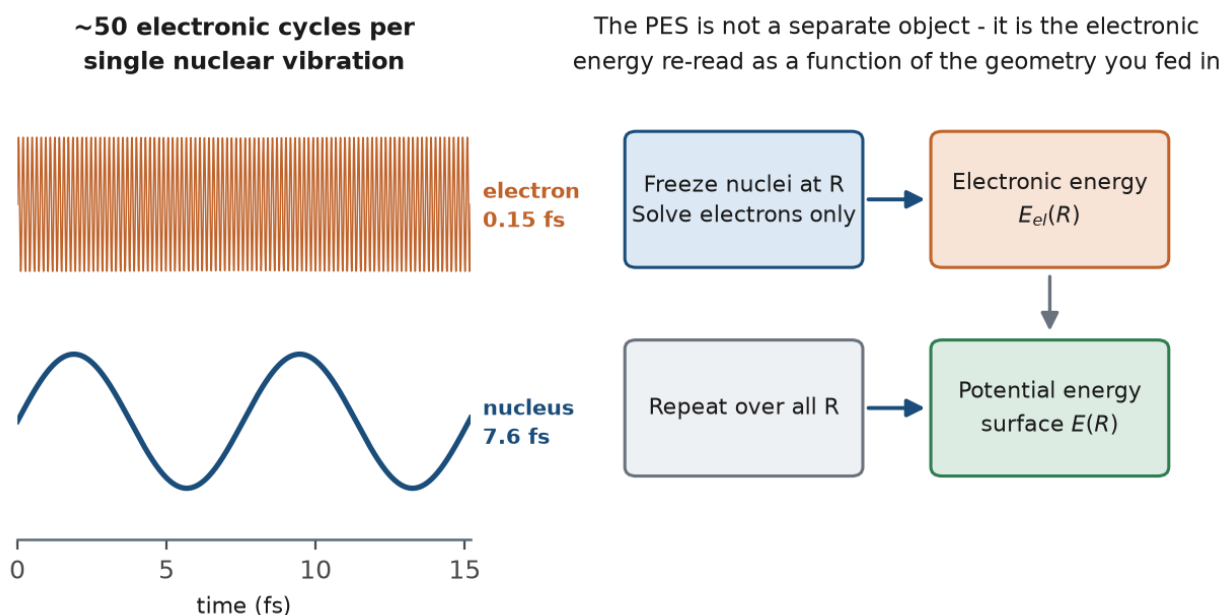


Figure 0.1 — The mass ratio buys a separation of timescales, and the separation of timescales is what lets you define $E(R)$ at all.

The consequence: from the electrons' point of view the nuclei are effectively stationary, and from the nuclei's point of view the electron cloud has always already re-equilibrated. This is called **adiabatic following**. It licenses the following move — freeze the nuclei at some arrangement $\mathbf{R}$, solve the electronic problem alone, and call the answer $E(\mathbf{R})$:

$$\Psi(\mathbf{r}, \mathbf{R}) \approx \psi_{\text{el}}(\mathbf{r}; \mathbf{R}) \cdot \chi_{\text{nuc}}(\mathbf{R})$$

Note the semicolon. In the electronic factor, $\mathbf{R}$ is a *parameter*, not a variable — you are not solving for the nuclei, you are solving for the electrons at a nuclear arrangement you chose by hand.

A sharper version of the same argument works in energy rather than time. Exciting an electron in H_2 costs about 0.41 Ha; one quantum of nuclear vibration carries about 0.02 Ha, roughly twenty times less. Nuclear motion simply does not have the energy budget to kick electrons out of their ground state.

3.4 The potential energy surface

Because you can compute an energy at *every* nuclear geometry, energy becomes a function of geometry. That function is the **potential energy surface** (PES). For a two-atom molecule there is only one geometric degree of freedom — the internuclear distance — so the surface collapses to a one-dimensional curve, the **bond dissociation curve**.

The PES is not a separate object you compute

It is the electronic energy, re-read as a function of the geometry you fed in. Every point on the curve is one complete, independent electronic-structure calculation. Phase 4 is nothing more than a loop that performs that calculation forty times.

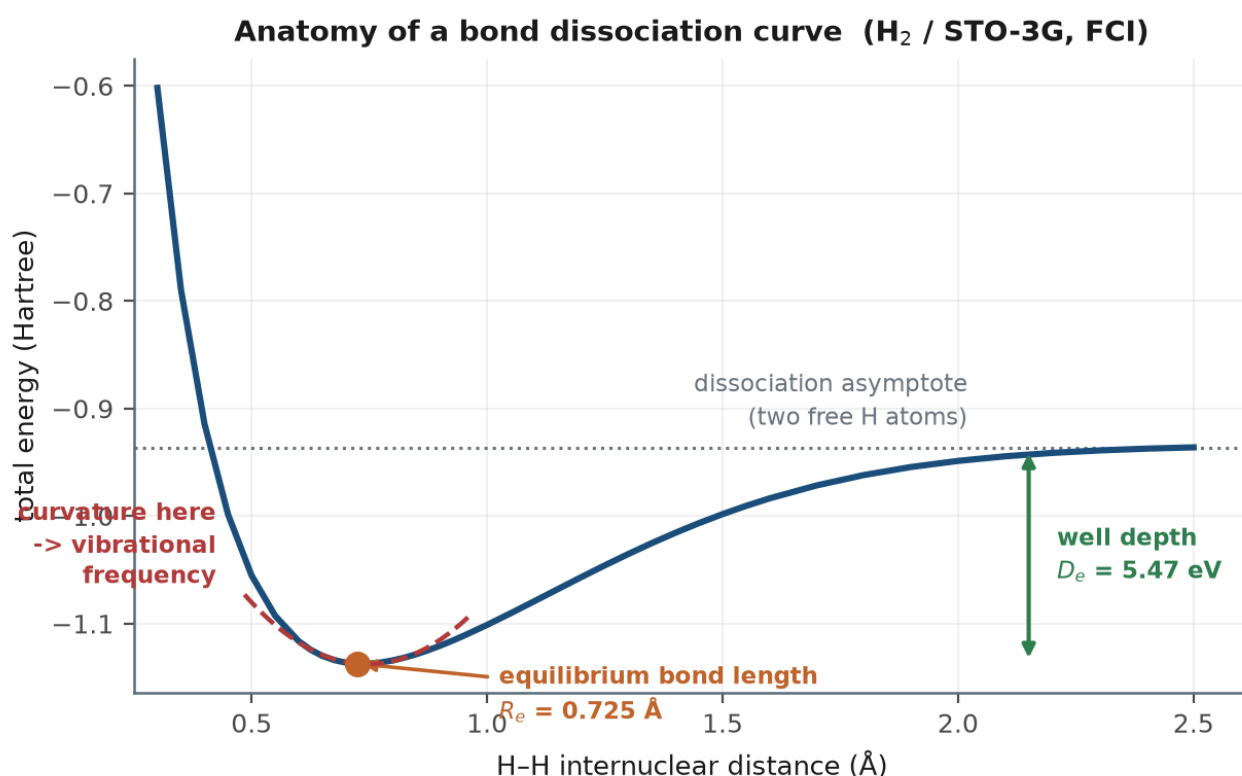


Figure 0.2 — The finished object, and the three physical constants Phase 5 extracts from it. This curve is real FCI/STO-3G data.

- **Equilibrium bond length R_e** — where the curve bottoms out. The molecule's natural size.

- **Well depth D_e** — how far the minimum sits below the flat asymptote. The energy needed to pull the molecule apart.
- **Harmonic vibrational frequency ω** — from the curvature at the bottom. A steep, narrow well is a stiff bond that vibrates fast. This is the quantity that connects your curve to infrared spectroscopy, i.e. to a real experiment.

4. The six stages of one energy evaluation

Every quantum-chemistry calculation — classical or quantum — passes through the same pipeline. Knowing where each phase of this project sits in that pipeline is the single most useful orientation you can have, because it tells you what is missing when something breaks.

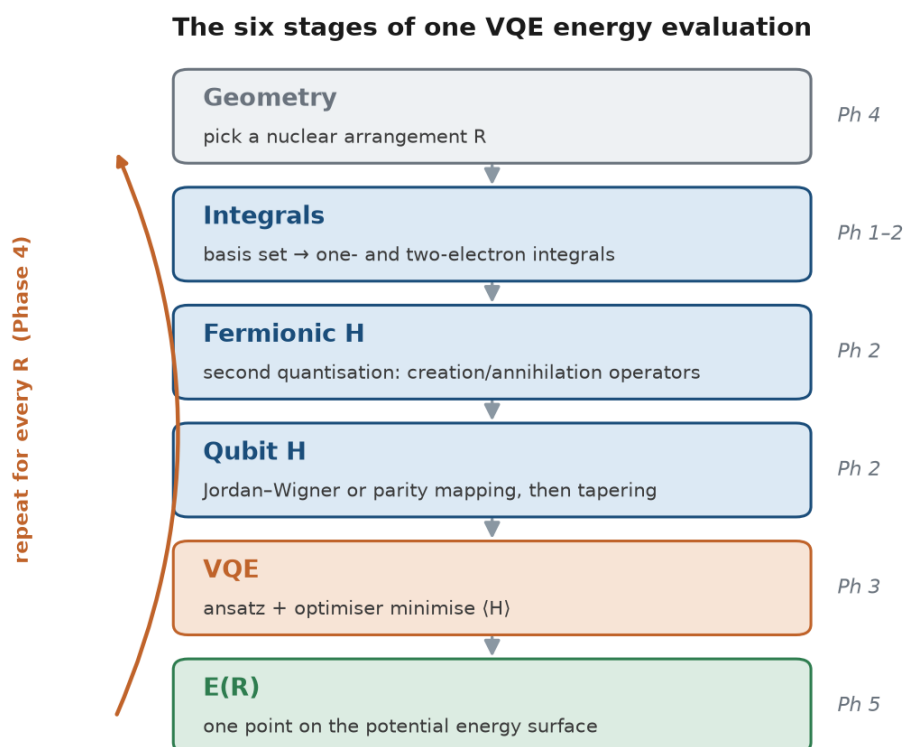


Figure 0.3 — The pipeline, annotated with the phase that covers each stage.

Two observations worth carrying forward. First, only **one** of the six stages is quantum. Stages 1–4 are classical quantum chemistry that has existed since the 1960s, and stage 6 is bookkeeping. Second, the loop back to the top is the whole point of the project — a single pass gives you one number, and forty passes give you a curve.

If you have built a VQE demo before

Most tutorials hand you the qubit Hamiltonian as a hardcoded list of Pauli coefficients, which silently skips stages 1 through 4. That is why such demos cannot change geometry, cannot change basis set, and cannot change the fermion-to-qubit mapping. Phase 2 of this project builds those stages properly, and it is the single biggest capability gain in the whole roadmap.

5. Why PennyLane rather than Qiskit

Both are excellent frameworks and this is not a quality judgement. The practical issue is that Qiskit 2.x moved VQE and the entire chemistry stack out of the core package into separately versioned `qiskit-algorithms` and `qiskit-nature` packages. Pinning a mutually compatible combination is a real and recurring time sink, and it has nothing to do with the physics you are trying to learn.

PennyLane's `qml.qchem` module is self-contained, its API has been stable across releases, and it calls PySCF underneath for the integrals. You get the whole pipeline from one import.

Component	Package	Role in this project
Quantum framework	pennylane	Circuits, ansätze, optimisers, qubit Hamiltonian construction
Quantum chemistry	pyscf	Molecular integrals, Hartree-Fock, FCI reference
Numerics	numpy, scipy	Arrays, curve fitting, polynomial fits in Phase 5
Plotting	matplotlib	The three-curve figure in Phase 5

PySCF is **not optional**. It supplies the classical reference you check VQE against, and it also drives the integral generation underneath PennyLane's chemistry module. Even if you never call it directly, it is running.

6. Build guide

Step 0.1 — Check your Python version

```
$ python3 --version
Python 3.14.4
```

Anything from 3.10 upward works. The reference build for this roadmap was verified on **Python 3.14.4 with PennyLane 0.45.1 and PySCF 2.14.0**. If you are on 3.9 or older, upgrade first — several dependencies have dropped support.

Step 0.2 — Create an isolated environment

Do not install into your system Python. Quantum-chemistry packages pull in large, version-sensitive dependency trees and you will want to be able to throw the whole thing away.

macOS / Linux

```
$ mkdir vqe-pes && cd vqe-pes
$ python3 -m venv .venv
$ source .venv/bin/activate
```

Windows PowerShell

```
> python -m venv .venv
> .venv\Scripts\activate
```

If you have `uv` installed it is considerably faster and the equivalent invocation is `uv venv .venv --python 3.13` followed by `uv pip install ...` in place of `pip install ...`. Either is fine.

Step 0.3 — Install the stack

```
$ pip install pennylane pyscf matplotlib numpy scipy
```

PySCF compiles native extensions and is the slowest item; expect a minute or two. On Apple Silicon it installs from a wheel and is quick. If PySCF fails to build, the usual cause is a missing C compiler — install the Xcode command line tools on macOS (`xcode-select --install`) or `build-essential` on Debian/Ubuntu.

Step 0.4 — Write the verification script

Do not proceed on the strength of `pip` exiting with status zero. Prove the stack works by computing something whose answer you already know. Save the following as `verify_env.py`:

Listing 0.1 — verify_env.py

```
"""verify_env.py - prove the environment works before writing any real code."""

import sys

print("python ", sys.version.split()[0])

import numpy
import scipy
import matplotlib
print("numpy ", numpy.__version__)
print("scipy ", scipy.__version__)
print("matplotlib", matplotlib.__version__)
```

```

import pennylane as qml
print("pennylane ", qml.version())

import pyscf
from pyscf import gto, scf, fci
print("pyscf      ", pyscf.__version__)
print()

# A real calculation with a known answer: H2 at its equilibrium geometry.
mol = gto.M(atom="H 0 0 0; H 0 0 0.735",
             basis="sto-3g",
             unit="Angstrom",
             verbose=0)

mf = scf.RHF(mol).run(verbose=0)
e_fci = fci.FCI(mf).kernel()[0]

print(f"nuclear repulsion {mol.energy_nuc():12.6f} Ha")
print(f"Hartree-Fock      {mf.e_tot:12.6f} Ha   (expect -1.116999)")
print(f"FCI                {e_fci:12.6f} Ha   (expect -1.137306)")
print(f"correlation energy {mf.e_tot - e_fci:12.6f} Ha")
print()

# And a qubit Hamiltonian, to prove the quantum half of the stack works.
import numpy as np
BOHR = 1.8897261254578281
coords = np.array([0.0, 0.0, 0.0,
                   0.0, 0.0, 0.735 * BOHR])
H, n_qubits = qml.qchem.molecular_hamiltonian(["H", "H"], coords,
                                              basis="sto-3g")

print(f"qubits required    {n_qubits}")
print(f"Pauli terms         {len(H.terms()[0])}")
print()
print("environment OK")

```

Step 0.5 — Run it

```
$ python verify_env.py
```

You should see output matching this:

Expected output

```

python      3.14.4
numpy       2.5.1
scipy       1.18.0
matplotlib  3.11.1
pennylane   0.45.1
pyscf       2.14.0

nuclear repulsion      0.719969 Ha
Hartree-Fock           -1.116999 Ha   (expect -1.116999)
FCI                    -1.137306 Ha   (expect -1.137306)
correlation energy      0.020307 Ha

qubits required        4
Pauli terms            15

environment OK

```

Library version numbers will drift; the **energies must not**. Those three numbers are properties of the physics, not of the software. If your Hartree-Fock energy is not -1.116999 Ha

to six decimal places, something is wrong with your geometry or basis specification and you must fix it before going further.

Step 0.6 — Lay out the project

Suggested layout — each phase adds one file

```
vqe-pes/  
├ .venv/  
├ verify_env.py           Phase 0  
├ classical.py           Phase 1 - HF and FCI reference  
├ hamiltonian.py         Phase 2 - geometry to qubit Hamiltonian  
├ vqe_single.py          Phase 3 - one geometry  
├ scan.py                Phase 4 - the loop  
├ analyse.py             Phase 5 - fitting and the figure  
├ compare_ansatz.py      Phase 6 - the extension  
└ data/  
  └ (scan output lands here)
```

One file per phase, each importable by the next. Resist the urge to put everything in a notebook: Phase 4 runs a loop you will want to restart, and Phase 6 re-uses Phase 3's machinery with one component swapped out.

7. Reference targets

Values to orient yourself. Treat them as a smell test, not as ground truth — **your own FCI result in your own basis is the authority**. Every number below was produced by the reference implementation and is reproducible with Listing 0.1.

Quantity	Value	Note
Spin orbitals (STO-3G)	4	2 spatial orbitals × 2 spins
Qubits, Jordan-Wigner	4	one per spin orbital
Qubits after tapering	1–2	using particle-number and spin symmetry
Pauli terms in H	15	Jordan-Wigner, at equilibrium
Equilibrium bond length	$\approx 0.735 \text{ \AA}$	experiment gives $0.741 \text{ \AA}$; close agreement is partly luck of the basis
Nuclear repulsion at R_e	0.719969 Ha	$1/R$ in atomic units — varies with geometry
Hartree-Fock at R_e	-1.116999 Ha	your mean-field reference
FCI at R_e	-1.137306 Ha	VQE should reproduce this to well under 1 mHa
Correlation energy	0.020307 Ha (0.553 eV)	the gap between HF and FCI — the quantity VQE exists to capture
Runtime, 40-point scan	1–3 min	laptop state-vector simulator, warm-started

Reading too much into STO-3G

Absolute energies and well depths from a minimal basis will disagree with experimental literature values, sometimes badly. **This is expected and is not a bug**. A minimal basis is a deliberately simplified question, and FCI is the exact answer to that simplified question. Compare VQE against your own FCI in the same basis. Never compare it against experiment.

8. What was done in this phase

No quantum code was written, and that is the point. What this phase established:

- **An isolated environment** with PennyLane, PySCF and the numerical stack, verified not by checking that imports succeed but by computing three energies with known values.
- **The Born-Oppenheimer approximation** as the licence to treat energy as a function of nuclear geometry. Without it, the phrase "the energy of H_2 at $1.2 \text{ \AA}$ " would not denote anything.
- **The potential energy surface** as the target object, and the three physical constants — equilibrium bond length, well depth, harmonic frequency — that Phase 5 will extract from it.
- **The six-stage pipeline**, and the fact that only one of those stages is quantum. This is the map you should return to whenever a later phase misbehaves: identify which stage is producing the wrong number before touching any code.

- **An honest framing of the deliverable.** VQE will not beat the classical solver here. The value is a verified implementation, and saying so plainly is a strength.
- **A set of reference targets**, all reproducible from Listing 0.1, against which every later phase can be checked.

9. Checklist before moving on

1. `python verify_env.py` runs to completion and prints `environment OK`.
2. Hartree-Fock reports -1.116999 Ha and FCI reports -1.137306 Ha.
3. The qubit Hamiltonian reports 4 qubits and 15 Pauli terms.
4. You can state, without looking it up, why energy can be treated as a function of bond length.
5. You can name which of the six pipeline stages is the quantum one.

Next

Phase 1 — The classical baseline. You build the oracle before you build the thing that needs checking against it. This is the phase most people skip, and skipping it is the most common way this project silently goes wrong.